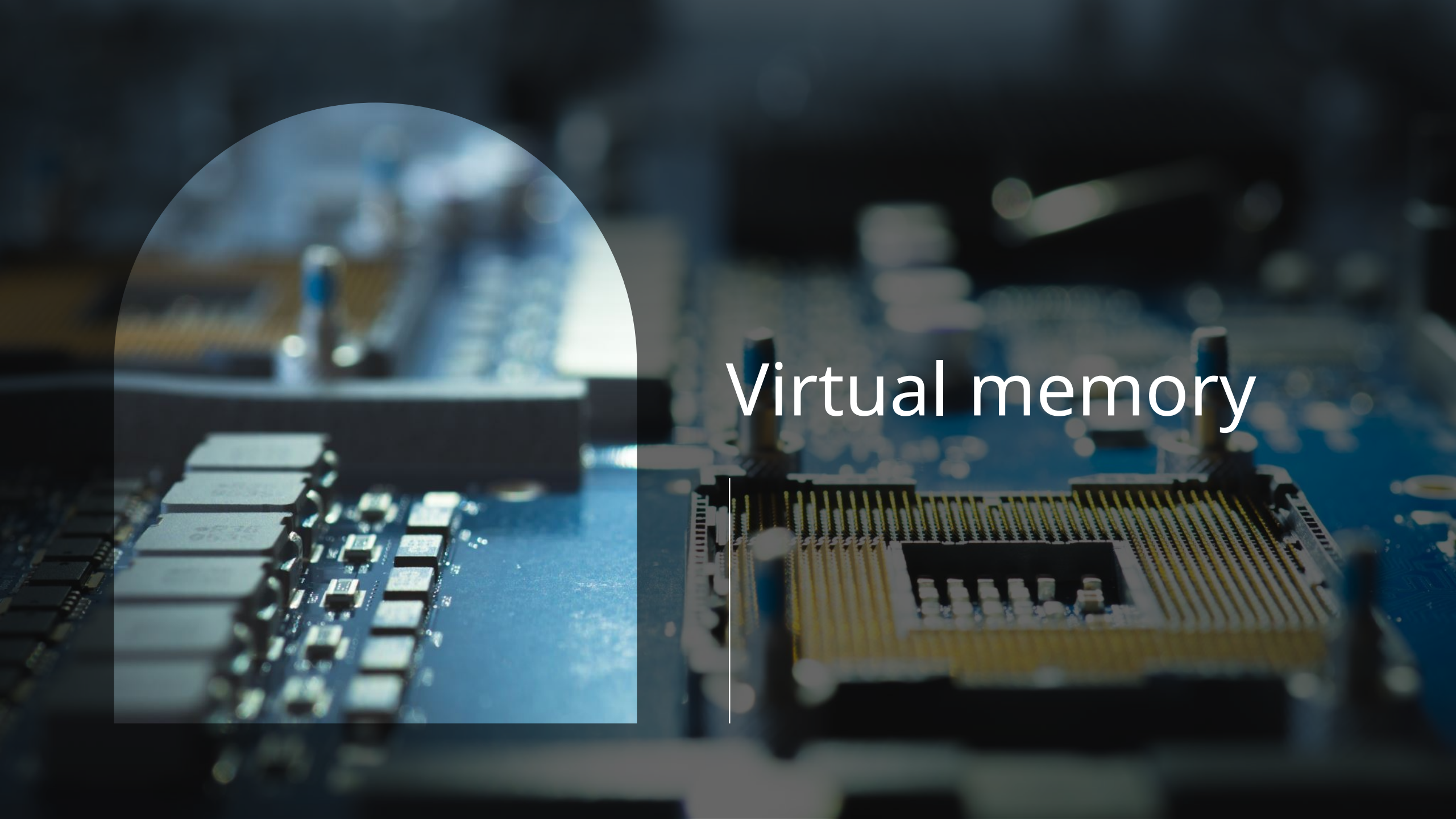
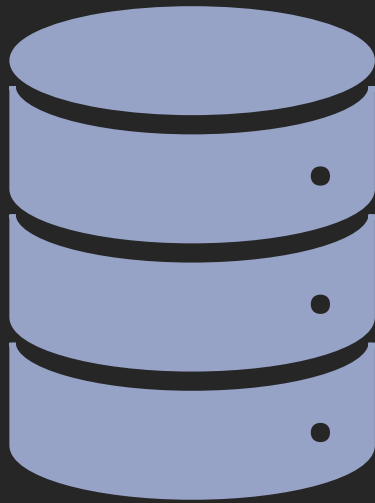




# Virtual memory



# Virtual memory

- Virtual memory is a technique that allows the execution of processes that may not be completely in memory.
- One major advantage of this scheme is that programs can be larger than physical memory.
- This technique frees programmers from the concerns of memory-storage limitations

# Virtual memory

The ability to execute a program that is only partially in memory would confer many benefits:

A program would no longer be constrained by the amount of physical memory that is available. Users would be able to write programs for an extremely large virtual-address space, simplifying the programming task.

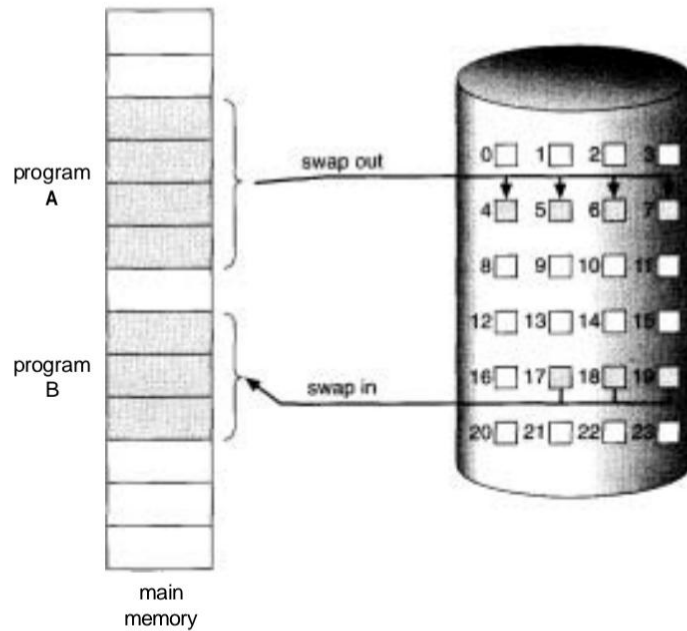
Because each user program could take less physical memory, more programs could be run at the same time, with a corresponding increase in CPU utilization and throughput, but with no increase in response time or turnaround time.

- Less I/O would be needed to load or swap each user program into memory, so each user program would run faster.
- **Virtual memory is commonly implemented by demand paging.**

# Demand paging

- Demand-paging system is similar to a paging system with swapping
  - Processes reside on secondary memory (which is usually a disk). When we want to execute a process, we swap it into memory. Rather than swapping the entire process into memory, however, we use a lazy swapper.
  - A lazy swapper never swaps a page into memory unless that page will be needed.
-

# Demand paging-Basic concept

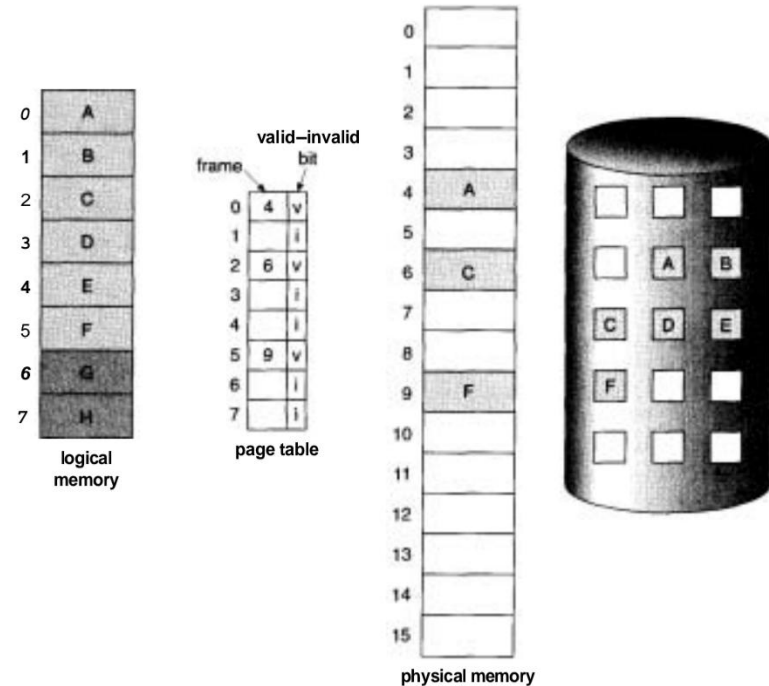


**Figure 10.2** Transfer of a paged memory to contiguous disk space.

# Demand paging-Basic concept

- With this scheme, we need some form of hardware support to distinguish between those pages that are in memory and those pages that are on the disk.
  - The valid-invalid bit scheme can be used for this purpose. This time, however, when this bit is set to “valid,” this value indicates that the associated page is both legal and in memory. If the bit is set to “invalid,” this value indicates that the page either is not valid (that is, not in the logical address space of the process), or is valid but is currently on the disk
-

# Demand paging-Basic concept

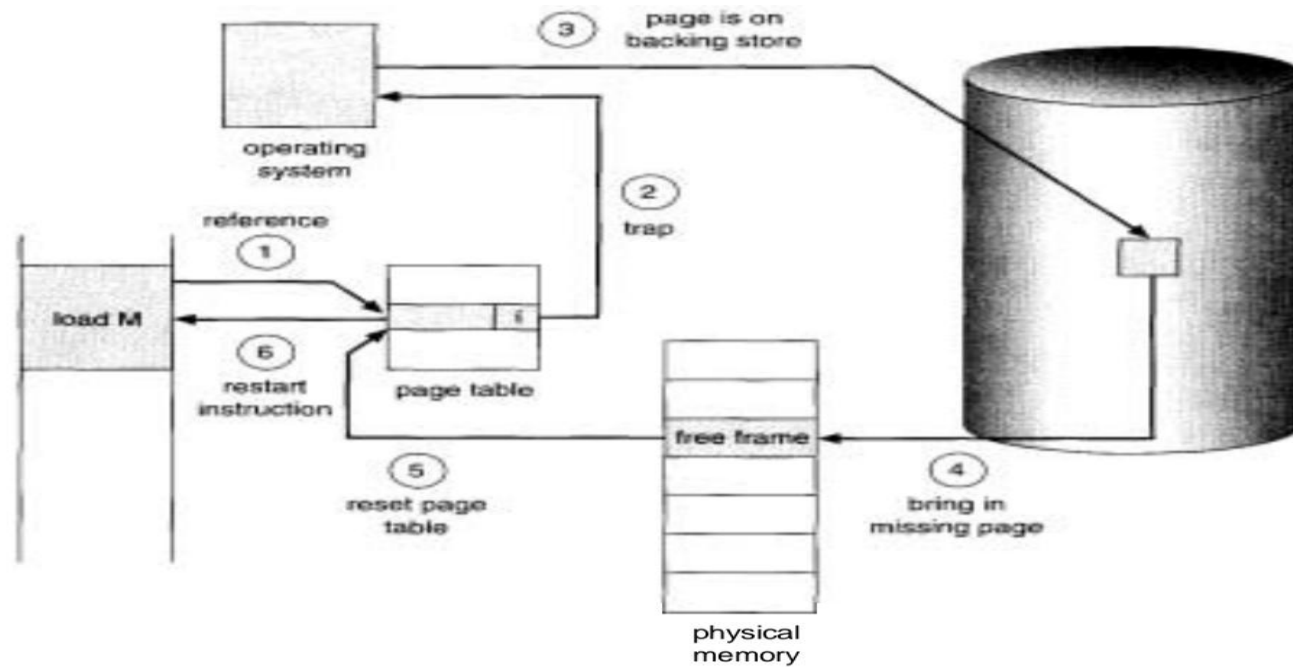


**Figure 10.3** Page table when some pages are not in main memory.

# Demand paging-Basic concept

- What happens if the process tries to access a page that was not brought into memory?
  - Access to a page marked invalid causes a page-fault trap. The paging hardware, in translating the address through the page table, will notice that the invalid bit is set, causing a trap to the operating system.
  - This trap is the result of the operating system's failure to bring the desired page into memory (in an attempt to minimize disk-transfer overhead and memory requirements), rather than an invalid address error as a result of an attempt to use an illegal memory address (such as an incorrect array subscript)
-

# Demand paging-Basic concept



**Figure 10.4** Steps in handling a page fault.

# Demand paging-Basic concept

1. We check an internal table (usually kept with the process control block) for this process, to determine whether the reference was a valid or invalid memory access
  2. If the reference was invalid, we terminate the process. If it was valid, but we have not yet brought in that page, we now page it in.
  3. We find a free frame (by taking one from the free-frame list, for example).
  4. We schedule a disk operation to read the desired page into the newly allocated frame.
  5. When the disk read is complete, we modify the internal table kept with the process and the page table to indicate that the page is now in memory.
  6. We restart the instruction that was interrupted by the illegal address trap. The process can now access the page as though it had always been in memory.
-

# Page replacement

1. Find the location of the desired page on the disk.
  2. Find a free frame:
    - a. If there is a free frame, use it.
    - b. If there is no free frame, use a page-replacement algorithm to select a victim frame.
    - c. Write the victim page to the disk; change the page and frame tables accordingly.
  3. Read the desired page into the (newly) free frame; change the page and frame tables.
  4. Restart the user process
-

# Page replacement

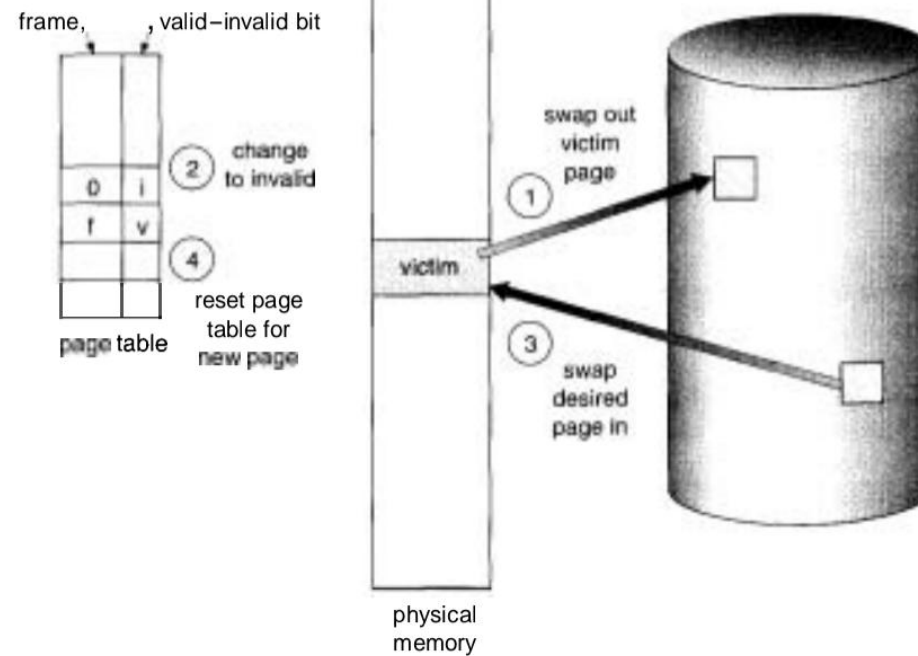


Figure 10.7 Page replacement.

# Page replacement

We evaluate an algorithm by running it on a particular string of memory references and computing the number of page faults. The string of memory references is called a reference string.

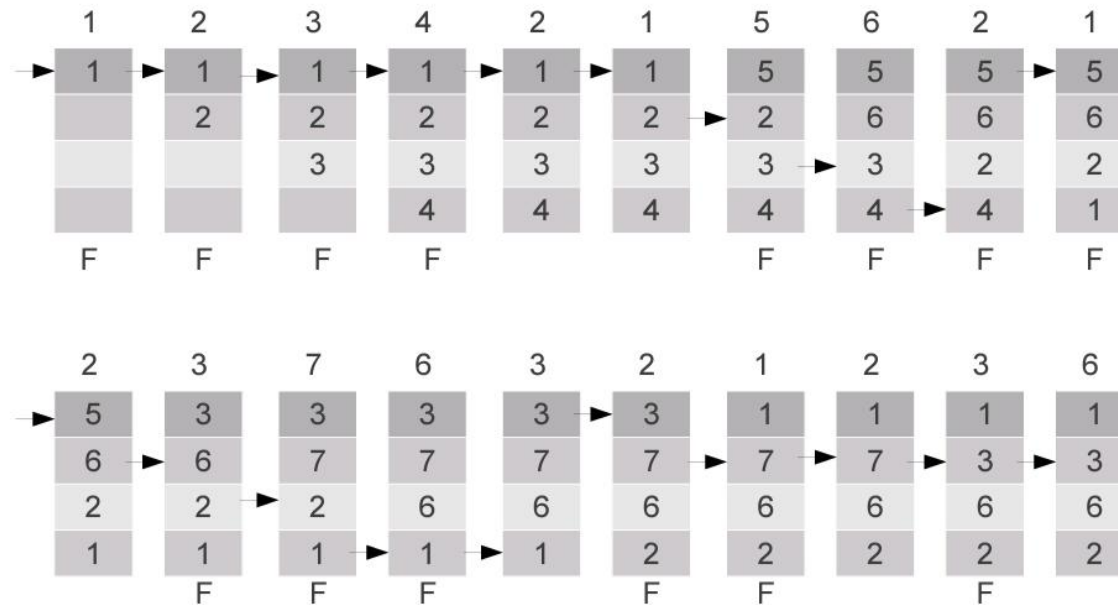
A **memory reference string** (also called a **page reference string** or simply a **reference string**) is a **sequence of memory addresses or page numbers** that represents the **order in which a program accesses memory**.

---

# FIFO Page Replacement

- Given page reference string:
  - 1,2,3,4,2,1,5,6,2,1,2,3,7,6,3,2,1,2,3,6
  - Compare the number of page faults for LRU, FIFO and Optimal page replacement algorithm if 4 page frames are there
  - A FIFO replacement algorithm associates with each page the time when that page was brought into memory. When a page must be replaced, the oldest page is chosen
-

# FIFO Page Replacement



# Belady's Anomaly

- **Bélády's anomaly** is the name given to the phenomenon where increasing the number of page frames results in an increase in the number of page faults for a given memory access pattern.

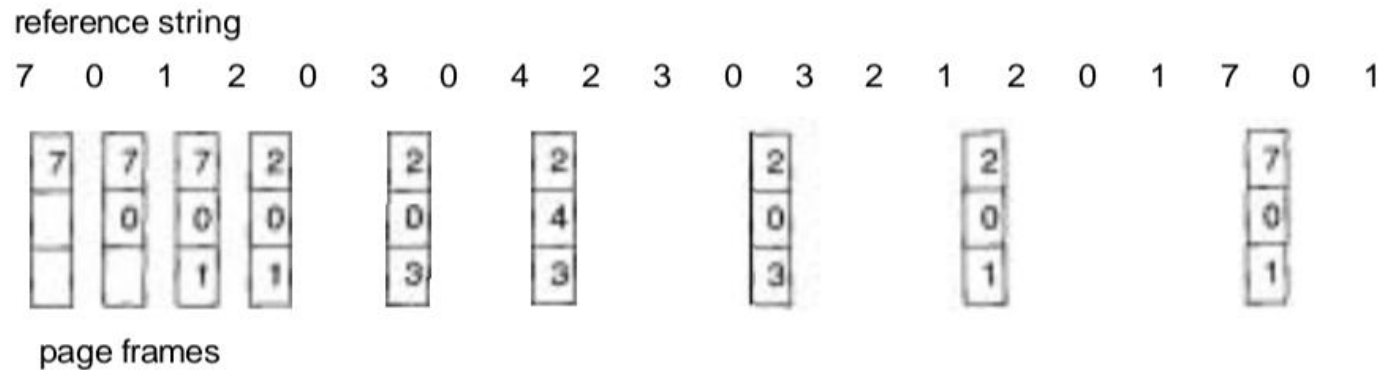
|    |    |    |    |    |    |    |   |   |    |    |   |
|----|----|----|----|----|----|----|---|---|----|----|---|
| 1  | 1  | 1  | 2  | 3  | 4  | 1  | 1 | 1 | 2  | 5  | 5 |
|    | 2  | 2  | 3  | 4  | 1  | 2  | 2 | 2 | 5  | 3  | 3 |
|    |    | 3  | 4  | 1  | 2  | 5  | 5 | 5 | 3  | 4  | 4 |
| PF | PF | PF | PF | PF | PF | PF | X | X | PF | PF | X |

# Belady's Anomaly

|    |    |    |    |   |   |    |    |    |    |    |    |
|----|----|----|----|---|---|----|----|----|----|----|----|
| 1  | 1  | 1  | 1  | 1 | 1 | 2  | 3  | 4  | 5  | 1  | 2  |
|    | 2  | 2  | 2  | 2 | 2 | 3  | 4  | 5  | 1  | 2  | 3  |
|    |    | 3  | 3  | 3 | 3 | 4  | 5  | 1  | 2  | 3  | 4  |
|    |    |    | 4  | 4 | 4 | 5  | 1  | 2  | 3  | 4  | 5  |
| PF | PF | PF | PF | X | X | PF | PF | PF | PF | PF | PF |

# Optimal Page Replacement

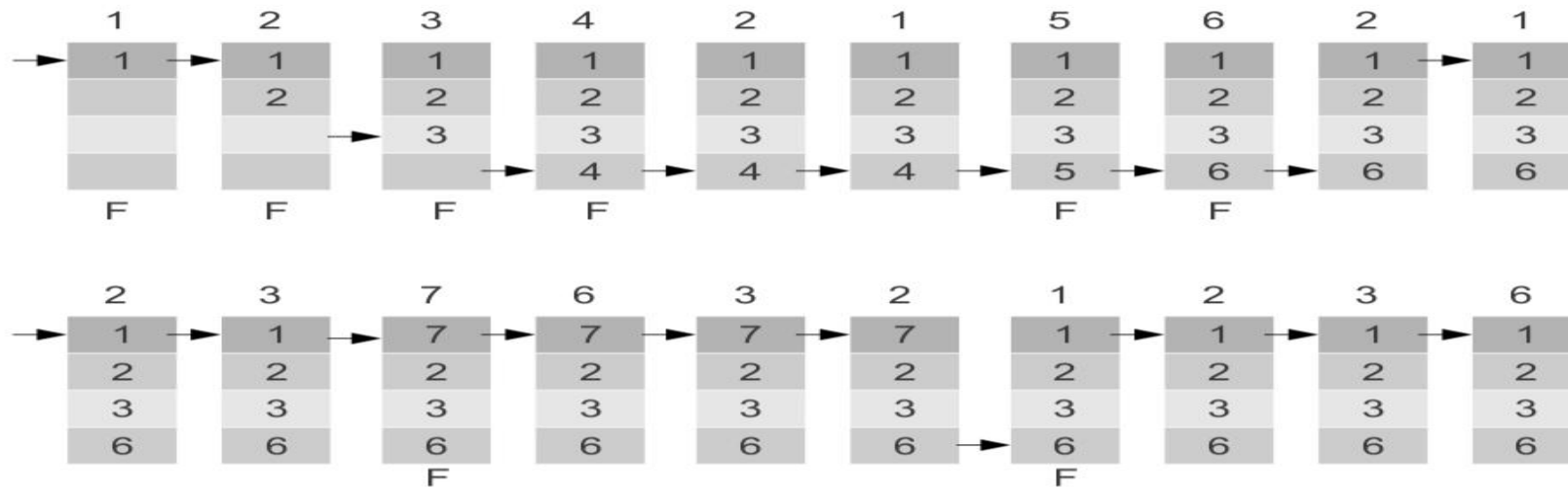
- Optimal page-replacement algorithm has the lowest page-fault rate of all algorithms, and will never suffer from Belady's anomaly.
- **Replace the page that will not be used for the longest period of time.**



**Figure 10.11** Optimal page-replacement algorithm.

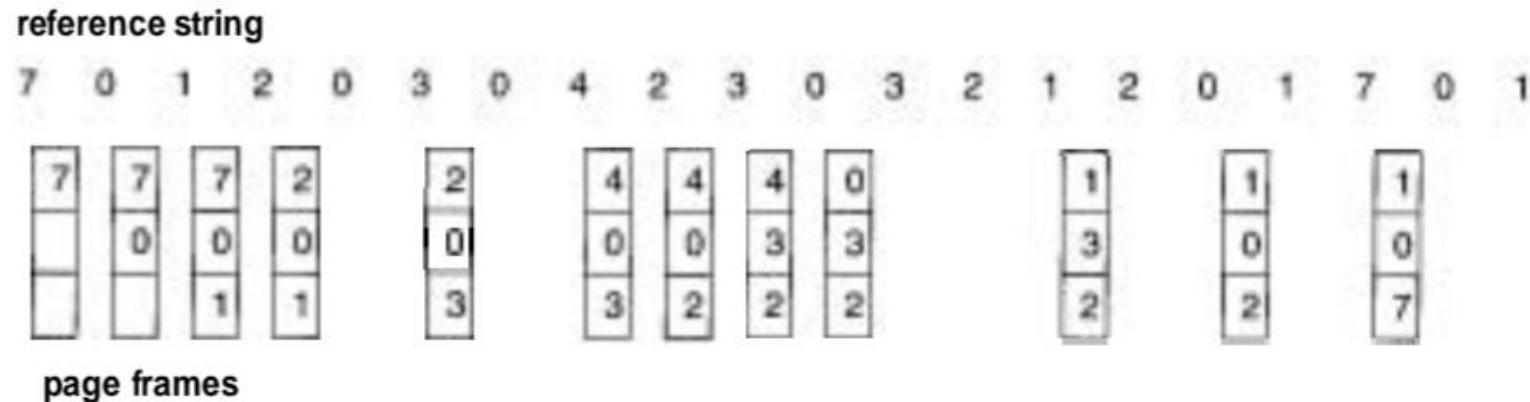
# Optimal Page Replacement

## Question 5 : 4 Frames, OPT



# Least Recently Used

- We use the recent past as an approximation of the near future, then we will replace the page that has not been used for the longest period of time (Figure 10.12). This approach is the least-recently-used (LRU) algorithm.



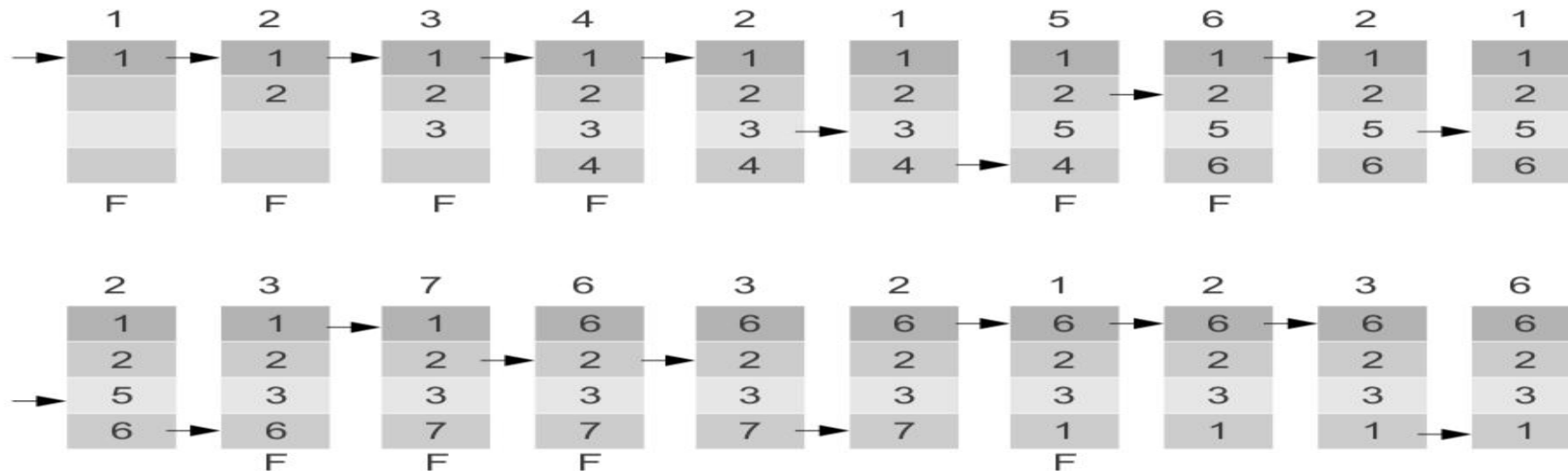
**Figure 10.12** LRU page-replacement algorithm.

# Least Recently Used

- When the reference to page 4 occurs, however, LRU replacement sees that, of the three frames in memory, page 2 was used least recently. The most recently used page is page 0, and just before that page 3 was used. Thus, the LRU algorithm replaces page 2, not knowing that page 2 is about to be used. When it then faults for page 2, the LRU algorithm replaces page 3 since, of the three pages in memory {0,3,4}, page 3 is the least recently used. Despite these problems, LRU replacement with 12 faults is still much better than FIFO replacement with 15.
-

# Least Recently Used

## Question 5 : 4 Frames, LRU



# Thrashing

- If a process does not have enough frames, the page-fault rate is very high
  - Suppose a process is allocated 5 frames and all pages are active. If there is a page fault now for that process, one of the 5 active frames is moved out and the required page is brought in. Since the page that is moved out is an active page, the moved out page may be needed immediately. There is a page fault again and another active page is moved out to bring in the required page. This leads to page faults again and again.
-

# Thrashing

- When the number of page faults becomes more, the system spends most of the time in paging activity (moving in and moving out pages). This results in low CPU utilization. Since the CPU utilization is low, the operating system thinks that there are not enough processes to use the CPU. Therefore, the operating system increases the degree of multiprogramming. That is, it adds another process to the system. This increases the paging activity even further. This results in thrashing, where a process spends more time on paging than executing.
-

# Thrashing

